

PANDUAN DEPLOYMENT LENGKAP

Aplikasi Presensi Siswa (Laravel) ke Google Cloud Platform (GCP) & Amazon Web Services (AWS)

Skripsi Berdampak — SMK Al Hafidz Leuwiliang

Sintia Sari — 0110222135 — STT Terpadu Nurul Fikri — 2026

GAMBARAN UMUM

Dokumen ini berisi panduan langkah demi langkah (step-by-step) untuk melakukan deployment aplikasi presensi siswa berbasis Laravel ke dua platform cloud, yaitu Google Cloud Platform (GCP) menggunakan Google App Engine dan Amazon Web Services (AWS) menggunakan Elastic Beanstalk. Panduan ini disusun khusus untuk pengguna Windows yang baru pertama kali menggunakan GCP.

ALUR KERJA KESELURUHAN:

- PERSIAPAN: Install tools & siapkan kode Laravel
- BAGIAN A — AWS (sudah punya akun): Deploy ke Elastic Beanstalk + RDS
- BAGIAN B — GCP (buat akun baru): Deploy ke App Engine + Cloud SQL
- MONITORING: Pasang Prometheus + Grafana di kedua platform
- PENGUJIAN: Jalankan load test dan catat hasilnya

⚠ PENTING — Baca sebelum mulai!

Minta file project Laravel dan file .sql database dari teman kamu SEBELUM memulai langkah apapun. Kamu butuh: (1) Folder project Laravel lengkap, (2) File dump database .sql, (3) File .env dari project tersebut.

BAGIAN 0 — PERSIAPAN TOOLS (WAJIB)

Lakukan semua langkah di bagian ini SATU KALI sebelum deploy ke manapun.

0.1 Install Git

1. Download Git untuk Windows

Buka browser, pergi ke <https://git-scm.com/download/win>, download dan install dengan pengaturan default.

2. Verifikasi instalasi

Buka Command Prompt (tekan Win+R → ketik cmd → Enter), lalu ketik perintah berikut:

```
git --version
```

Hasil yang muncul harus seperti: git version 2.x.x

0.2 Install PHP 8.2 & Composer

1. Download Laragon (cara termudah di Windows)

Pergi ke <https://laragon.org/download> → pilih Laragon Full → install. Laragon otomatis menyertakan PHP 8.2, MySQL, dan Composer.

2. Verifikasi PHP dan Composer

Buka terminal Laragon (klik kanan icon Laragon → Terminal), lalu ketik:

```
php --version  
composer --version
```

Keduanya harus menampilkan versi yang terinstall.

0.3 Siapkan Project Laravel

1. Terima file dari teman

Minta teman kamu mengirimkan: folder project Laravel (zip), file .sql database, dan file .env. Simpan di folder yang mudah diakses, misal: C:\presensi-app\

2. Extract dan install dependensi

Buka terminal Laragon, masuk ke folder project:

```
cd C:\presensi-app  
composer install  
cp .env.example .env  
php artisan key:generate
```

3. Edit file .env

Buka file .env dengan Notepad++ atau VS Code. Pastikan pengaturan database sesuai dengan yang akan digunakan di cloud (akan diupdate nanti).

⚠ Catatan .env

Jangan upload file .env ke Git! File ini berisi password database. Pastikan .env sudah ada di dalam file .gitignore sebelum push.

0.4 Buat Repository Git

1. Inisialisasi Git di folder project

```
cd C:\presensi-app
git init
git add .
git commit -m "Initial commit - aplikasi presensi"
```

2. Buat akun GitHub (jika belum ada)

Pergi ke <https://github.com> → Sign Up. Buat repository baru bernama 'presensi-smk-alhafidz', pilih Private.

3. Push ke GitHub

```
git remote add origin https://github.com/USERNAME/presensi-smk-
alhafidz.git
git branch -M main
git push -u origin main
```

✓ Persiapan Selesai!

Setelah langkah 0.1–0.4 selesai, kamu sudah siap deploy ke AWS dan GCP.

BAGIAN A — DEPLOY KE AWS (ELASTIC BEANSTALK + RDS)

Estimasi waktu: 60–90 menit | Biaya: ~\$40/bulan (Free Tier tersedia untuk akun baru)

A.1 Install AWS CLI & EB CLI

1. Download AWS CLI

Buka <https://awscli.amazonaws.com/AWSCLIV2.msi> → install dengan klik Next terus → Finish.

2. Konfigurasi AWS CLI

Buka Command Prompt, ketik:

```
aws configure
```

Kamu akan diminta mengisi empat data:

- AWS Access Key ID → ambil dari AWS Console > IAM > Users > Security credentials > Create access key
- AWS Secret Access Key → muncul saat buat access key (simpan baik-baik!)
- Default region name → ketik: ap-southeast-1
- Default output format → ketik: json

3. Install EB CLI (Elastic Beanstalk CLI)

Jalankan di Command Prompt:

```
pip install awsebcli
```

⚠ Jika pip tidak dikenal

Install Python dulu dari <https://python.org/downloads> → saat install centang 'Add Python to PATH' → restart Command Prompt.

4. Verifikasi EB CLI

```
eb --version
```

A.2 Buat Database RDS MySQL

Langkah ini membuat database MySQL terkelola di AWS yang akan digunakan oleh aplikasi Laravel.

1. Buka AWS Console

Login ke <https://console.aws.amazon.com> → di kolom pencarian ketik 'RDS' → klik RDS.

2. Buat database baru

Klik tombol 'Create database', lalu isi pengaturan berikut:

```
Engine type      : MySQL
Version         : MySQL 8.0
Template        : Free tier (atau Dev/Test)
DB instance ID  : presensi-db
Master username : admin
Master password : [buat password kuat, catat!]
DB instance class : db.t3.micro
Storage type    : gp2 | 20 GB
Public access   : Yes (sementara, untuk import data)
VPC security group : Create new
```

3. Tunggu database aktif

Status akan berubah dari 'Creating' → 'Available'. Proses ini 5–10 menit. Catat nilai 'Endpoint' yang muncul, contoh: `presensi-db.xxxxx.ap-southeast-1.rds.amazonaws.com`

4. Import database dari file .sql

Download MySQL Workbench dari <https://dev.mysql.com/downloads/workbench/> → buka → klik '+' untuk koneksi baru → isi:

```
Hostname : [endpoint RDS yang dicatat tadi]
Port     : 3306
Username : admin
Password : [password yang dibuat tadi]
```

Setelah konek, klik menu Server → Data Import → pilih file .sql dari teman → klik Start Import.

A.3 Konfigurasi Laravel untuk AWS

1. Update file .env

Buka file .env di folder project, update bagian database:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://[URL-elastic-beanstalk-nanti]

DB_CONNECTION=mysql
DB_HOST=[endpoint RDS kamu]
DB_PORT=3306
DB_DATABASE=[nama database]
DB_USERNAME=admin
DB_PASSWORD=[password RDS kamu]
```

2. Buat file konfigurasi Nginx untuk Elastic Beanstalk

Buat folder `.ebextensions` di root project, lalu buat file `.ebextensions/nginx.config`:

```
files:
  "/etc/nginx/conf.d/proxy.conf":
    mode: '000644'
    owner: root
    group: root
    content: |
      client_max_body_size 10M;
```

3. Buat file `.ebextensions/laravel.config`

File ini mengatur environment PHP dan menjalankan migrasi otomatis:

```
option_settings:
  aws:elasticbeanstalk:container:php:phpini:
    document_root: /public
    memory_limit: 256M
container_commands:
  01_migrate:
    command: 'php artisan migrate --force'
    leader_only: true
```

4. Tambahkan storage link

Tambahkan ke `.ebextensions/laravel.config` bagian `container_commands`:

```
02_storage_link:
  command: 'php artisan storage:link'
  leader_only: true
```

A.4 Deploy ke Elastic Beanstalk

1. Inisialisasi Elastic Beanstalk di folder project

Buka Command Prompt, masuk ke folder project:

```
cd C:\presensi-app
eb init
```

Jawab pertanyaan yang muncul:

- Select a default region → pilih 10 (ap-southeast-1 : Asia Pacific (Singapore))
- Application Name → presensi-smk-alhafidz
- It appears you are using PHP. Is this correct? → Y
- Select a platform branch → PHP 8.2 running on 64bit Amazon Linux 2023
- Do you want to set up SSH? → n (pilih n dulu)

2. Buat environment production

```
eb create presensi-production --instance-type t3.small --single
```

Proses ini 10–15 menit. Tunggu hingga selesai. Kamu akan melihat log berjalan di terminal.

3. Set environment variables di AWS

Jalankan perintah ini untuk mengatur variabel `.env` di cloud (jangan masukkan `.env` langsung):

```
eb setenv APP_ENV=production APP_DEBUG=false \  
DB_CONNECTION=mysql \  
DB_HOST=[endpoint-rds] \  
DB_DATABASE=[nama-db] \  
DB_USERNAME=admin \  
DB_PASSWORD=[password-rds] \  
APP_KEY=[isi dari .env lokal kamu]
```

4. Buka aplikasi

```
eb open
```

Browser akan membuka URL aplikasi kamu secara otomatis. Jika tampil halaman Laravel, deploy berhasil!

✓ AWS Deploy Berhasil!

Catat URL Elastic Beanstalk kamu (format: `http://presensi-production.ap-southeast-1.elasticbeanstalk.com`). URL ini dibutuhkan untuk pengujian.

BAGIAN B — DEPLOY KE GCP (APP ENGINE + CLOUD SQL)

Estimasi waktu: 60–90 menit | Biaya: ~\$42/bulan | Google memberi kredit \$300 untuk akun baru!

B.1 Buat Akun Google Cloud

1. Daftar Google Cloud

Buka <https://cloud.google.com> → klik 'Get started for free' → login dengan akun Google → isi data kartu kredit (tidak ditagih selama masa trial, hanya untuk verifikasi).

⚠ Kartu Kredit

Google memberikan kredit \$300 gratis untuk 90 hari pertama. Kamu tidak akan ditagih secara otomatis kecuali sengaja upgrade ke paid account.

2. Buat project baru

Di Google Cloud Console (console.cloud.google.com), klik dropdown project di pojok kiri atas → 'New Project':

```
Project name : presensi-smk-alhafidz
Project ID   : presensi-smk-alhafidz (atau yang tersedia)
```

3. Aktifkan billing

Klik menu Navigation (☰) → Billing → pastikan akun billing terhubung ke project.

B.2 Install Google Cloud SDK (gcloud CLI)

1. Download Google Cloud SDK

Buka <https://cloud.google.com/sdk/docs/install> → pilih Windows → download file installer (.exe) → jalankan installer → Next terus → Finish.

2. Login dan set project

Buka Command Prompt baru (yang sudah mengenali gcloud), lalu:

```
gcloud init
```

Jawab pertanyaan yang muncul:

- Would you like to log in? → Y → browser akan terbuka → login akun Google kamu
- Pick cloud project → pilih presensi-smk-alhafidz

- Do you want to configure a default Compute Region? → Y → pilih 23 (asia-southeast2-a Jakarta)

3. Verifikasi

```
gcloud config list
```

Pastikan project dan account sudah benar.

B.3 Aktifkan API yang Diperlukan

1. Aktifkan semua API sekaligus

Jalankan di Command Prompt:

```
gcloud services enable appengine.googleapis.com  
gcloud services enable sqladmin.googleapis.com  
gcloud services enable cloudbuild.googleapis.com
```

Tunggu beberapa detik hingga setiap perintah selesai.

B.4 Buat Database Cloud SQL

1. Buat instance Cloud SQL

```
gcloud sql instances create presensi-db \  
  --database-version=MYSQL_8_0 \  
  --tier=db-f1-micro \  
  --region=asia-southeast2 \  
  --storage-size=20GB \  
  --storage-type=SSD \  
  --root-password=[buat-password-kuat]
```

Proses ini 5–10 menit. Tunggu hingga selesai.

2. Buat database di dalam instance

```
gcloud sql databases create presensi_db --instance=presensi-db
```

3. Buat user database

```
gcloud sql users create laraveluser \  
  --instance=presensi-db \  
  --password=[password-untuk-laravel]
```

4. Catat Connection Name

```
gcloud sql instances describe presensi-db --  
format='value(connectionName) '
```

Hasilnya berbentuk: presensi-smk-alhafidz:asia-southeast2:presensi-db → CATAT ini, akan dipakai di app.yaml.

5. Import database .sql ke Cloud SQL

Aktifkan koneksi IP sementara untuk import:

```
gcloud sql instances patch presensi-db --authorized-networks=0.0.0.0/0
```

Lalu buka MySQL Workbench → koneksi baru:

```
Hostname : [IP public Cloud SQL - lihat di GCP Console > SQL > instance]
Port      : 3306
Username  : laraveluser
Password  : [password yang dibuat tadi]
```

Import file .sql: Server → Data Import → pilih file .sql → Start Import.

Setelah selesai, matikan authorized networks (keamanan):

```
gcloud sql instances patch presensi-db --no-authorized-networks
```

B.5 Konfigurasi Laravel untuk GCP

1. Buat file app.yaml di root folder project

File ini adalah konfigurasi utama Google App Engine:

```
runtime: php82

env_variables:
  APP_ENV: production
  APP_DEBUG: false
  APP_KEY: [isi dari .env lokal kamu]
  DB_CONNECTION: mysql
  DB_SOCKET: /cloudsql/[CONNECTION_NAME_TADI]
  DB_DATABASE: presensi_db
  DB_USERNAME: laraveluser
  DB_PASSWORD: [password-laravel]

beta_settings:
  cloud_sql_instances: [CONNECTION_NAME_TADI]

handlers:
- url: /*
  script: auto
  secure: always
```

⚠ Ganti Placeholder

Ganti [CONNECTION_NAME_TADI] dengan nilai dari langkah B.4 no.4, contoh: presensi-smk-alhafidz:asia-southeast2:presensi-db

2. Buat file .gcloudignore

Agar file tidak perlu di-upload ke cloud:

```
.git
.gitignore
.env
node_modules/
storage/logs/*
```

3. Inisialisasi App Engine

Jalankan SATU KALI saja:

```
gcloud app create --region=asia-southeast2
```

B.6 Deploy ke App Engine

1. Pastikan sudah di folder project

```
cd C:\presensi-app
```

2. Deploy!

```
gcloud app deploy app.yaml --quiet
```

Proses ini 5–15 menit tergantung ukuran project. Kamu akan melihat progress upload dan build.

3. Buka aplikasi

```
gcloud app browse
```

Browser akan membuka URL aplikasi. Format URL: <https://presensi-smk-alhafidz.as.r.appspot.com>

4. Jalankan migrasi (jika belum otomatis)

```
gcloud app instances ssh [INSTANCE_ID] -- php artisan migrate --force
```

✓ GCP Deploy Berhasil!

Catat URL App Engine kamu. Format: [https://\[project-id\].as.r.appspot.com](https://[project-id].as.r.appspot.com) — URL ini dibutuhkan untuk pengujian.

BAGIAN C — PASANG MONITORING (PROMETHEUS + GRAFANA)

Prometheus dan Grafana dipasang di VM terpisah yang dapat mengakses kedua platform cloud. Cara termudah adalah menggunakan EC2 di AWS sebagai server monitoring.

C.1 Buat VM untuk Monitoring di AWS

1. Buka EC2 Console

AWS Console → ketik EC2 di pencarian → Launch Instance:

```
Name           : monitoring-server
AMI            : Ubuntu Server 22.04 LTS
Instance type  : t3.micro (Free Tier eligible)
Key pair       : Create new → monitoring-key → Download .pem
Security group : Allow SSH (22), HTTP (80), port 3000, 9090, 9091
```

2. Koneksi ke VM via SSH

Download PuTTY dari <https://putty.org> → konversi .pem ke .ppk menggunakan PuTTYgen → koneksi ke IP public VM.

Atau gunakan Windows Terminal dengan perintah:

```
ssh -i monitoring-key.pem ubuntu@[IP-PUBLIC-VM]
```

C.2 Install Prometheus

1. Jalankan perintah berikut di dalam VM (via SSH):

```
sudo apt update && sudo apt upgrade -y
wget
https://github.com/prometheus/prometheus/releases/download/v2.51.0/prometheus-2.51.0.linux-amd64.tar.gz
tar -xvf prometheus-2.51.0.linux-amd64.tar.gz
sudo mv prometheus-2.51.0.linux-amd64 /opt/prometheus
sudo useradd --no-create-home --shell /bin/false prometheus
sudo chown -R prometheus:prometheus /opt/prometheus
```

2. Buat konfigurasi Prometheus

```
sudo nano /opt/prometheus/prometheus.yml
```

Isi file dengan konfigurasi berikut (ganti URL dengan URL aplikasi kamu):

```
global:
  scrape_interval: 15s

scrape_configs:
```

```
- job_name: 'aws_app'
  static_configs:
    - targets: ['[URL-AWS-KAMU]:80']
    metrics_path: '/metrics'

- job_name: 'gcp_app'
  static_configs:
    - targets: ['[URL-GCP-KAMU]:443']
    metrics_path: '/metrics'
  scheme: https
```

⚠ Endpoint /metrics

Kamu perlu menambahkan package prometheus-php ke project Laravel. Jalankan: `composer require promphp/prometheus_client_php` — lalu buat route /metrics di routes/web.php.

3. Buat service Prometheus agar berjalan otomatis

```
sudo nano /etc/systemd/system/prometheus.service
```

Isi dengan:

```
[Unit]
Description=Prometheus
After=network.target

[Service]
User=prometheus
ExecStart=/opt/prometheus/prometheus \
  --config.file=/opt/prometheus/prometheus.yml \
  --storage.tsdb.path=/opt/prometheus/data
Restart=always

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable prometheus
sudo systemctl start prometheus
```

Akses Prometheus di browser: [http://\[IP-VM\]:9090](http://[IP-VM]:9090)

C.3 Install Grafana

1. Install Grafana di VM yang sama

```
sudo apt install -y software-properties-common
sudo add-apt-repository 'deb https://packages.grafana.com/oss/deb stable main'
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
sudo apt update && sudo apt install grafana -y
sudo systemctl enable grafana-server
sudo systemctl start grafana-server
```

2. Akses Grafana

Buka browser: `http://[IP-VM]:3000`

- Username default: admin
- Password default: admin → kamu akan diminta ganti saat login pertama

3. Tambahkan Prometheus sebagai Data Source

Di Grafana: Configuration (ikon gear) → Data Sources → Add data source → pilih Prometheus → isi URL: `http://localhost:9090` → Save & Test.

4. Import dashboard

Di Grafana: Dashboards → Import → masukkan ID: 14057 (Laravel Monitoring) → Load → pilih Prometheus datasource → Import.

✓ Monitoring Siap!

Prometheus berjalan di port 9090, Grafana di port 3000. Kamu bisa pantau performa kedua aplikasi secara real-time dari satu dashboard.

BAGIAN D — PENGUJIAN PERFORMA

Pengujian dilakukan menggunakan Apache JMeter yang dijalankan dari laptop Windows kamu. JMeter akan mengirim request simultan ke kedua URL aplikasi (AWS dan GCP) secara bergantian.

D.1 Install Apache JMeter

1. Install Java (prerequisite JMeter)

Download dari <https://adoptium.net> → pilih Java 17 LTS → installer Windows → install dengan default.

2. Download JMeter

Buka https://jmeter.apache.org/download_jmeter.cgi → download apache-jmeter-5.x.x.zip → extract ke C:\jmeter\

3. Jalankan JMeter

Buka C:\jmeter\bin → double-click jmeter.bat → tunggu GUI terbuka.

D.2 Buat Test Plan JMeter

1. Buat Thread Group

Di JMeter: klik kanan Test Plan → Add → Threads (Users) → Thread Group → isi:

```
Number of Threads (users) : 100
Ramp-up period (seconds)  : 10
Loop Count                 : 3
```

2. Tambahkan HTTP Request

Klik kanan Thread Group → Add → Sampler → HTTP Request → isi URL aplikasi AWS:

```
Protocol   : http
Server     : [URL-AWS-tanpa-http]
Path       : /login
Method     : GET
```

3. Tambahkan listeners untuk mencatat hasil

Klik kanan Thread Group → Add → Listener → tambahkan:

- Summary Report — untuk melihat rata-rata response time & throughput
- Response Time Graph — untuk grafik response time
- View Results Tree — untuk debug jika ada error

4. Jalankan pengujian untuk AWS

Klik tombol Run (▶) di toolbar → tunggu selesai → catat hasil di Summary Report.

5. Ganti URL ke GCP dan ulangi

Ubah HTTP Request URL ke URL GCP → jalankan lagi dengan setting yang sama → catat hasilnya.

D.3 Skenario Pengujian Lengkap

Ulangi langkah D.2 untuk setiap skenario berikut. Catat semua hasilnya ke tabel Excel untuk dimasukkan ke Bab IV skripsi.

Skenario	Thread (Users)	Ramp-up	Loop	Yang Diukur
S-01	10	5 detik	3x	Baseline / normal load
S-02	50	10 detik	3x	Medium load
S-03	100	10 detik	3x	Peak load (kondisi nyata)
S-04	100	10 detik	6x (10 menit)	Stress test / stabilitas

D.4 Data yang Perlu Dicatat

Dari JMeter Summary Report, catat kolom-kolom berikut untuk SETIAP skenario dan SETIAP platform:

- Average (ms) → Response Time rata-rata
- 90th pct (ms) → Persentil 90 response time
- 95th pct (ms) → Persentil 95 response time (P95)
- Min (ms) → Latency minimum
- Max (ms) → Latency maksimum
- Throughput → req/sec
- Error % → Error rate

⚠ Cara baca data dari Grafana

Setelah pengujian selesai, buka Grafana di [http://\[IP-VM\]:3000](http://[IP-VM]:3000) → buka dashboard → kamu bisa screenshot grafik CPU usage, memory, dan network untuk dilampirkan di Bab IV skripsi.

BAGIAN E — ESTIMASI BIAYA

E.1 Cara Hitung Biaya AWS

1. Buka AWS Pricing Calculator

Pergi ke <https://calculator.aws/pricing/2/home> → klik 'Create estimate'

2. Tambahkan layanan Elastic Beanstalk (EC2)

Search 'EC2' → Add → isi: Region ap-southeast-1, instance t3.small, 1 instance, 730 jam/bulan

3. Tambahkan RDS MySQL

Search 'RDS' → Add → isi: MySQL, db.t3.micro, Single-AZ, 20 GB storage, 730 jam/bulan

4. Tambahkan estimasi transfer data

Search 'Data Transfer' → Add → estimasi 50 GB outbound per bulan

5. Lihat total

Klik 'View summary' → catat total USD/month → export sebagai PDF atau screenshot untuk lampiran skripsi

E.2 Cara Hitung Biaya GCP

1. Buka Google Cloud Pricing Calculator

Pergi ke <https://cloud.google.com/products/calculator>

2. Tambahkan App Engine

Klik App Engine → pilih Standard Environment, F2 instance, estimasi jam aktif per bulan

3. Tambahkan Cloud SQL

Klik Cloud SQL → MySQL, db-f1-micro, 20 GB SSD, region asia-southeast2

4. Tambahkan Networking

Estimasi 50 GB egress per bulan → tambahkan ke kalkulasi

5. Lihat estimasi total

Klik 'Estimate' → catat total → compare dengan AWS

CHECKLIST AKHIR — SEBELUM MULAI TULIS DATA KE BAB IV

Centang semua item berikut sebelum mengisi data ke Bab IV skripsi:

PERSIAPAN:

- ☐ File project Laravel sudah diterima dari teman
- ☐ File .sql database sudah diterima
- ☐ Git, PHP, Composer, Laragon sudah terinstall
- ☐ Repository GitHub sudah dibuat

AWS:

- ☐ AWS CLI & EB CLI terinstall dan terkonfigurasi
- ☐ RDS MySQL sudah dibuat dan aktif
- ☐ Database .sql sudah diimport ke RDS
- ☐ File .ebextensions sudah dibuat
- ☐ eb create berhasil — environment aktif
- ☐ URL AWS bisa diakses di browser dan aplikasi berjalan

GCP:

- ☐ Akun Google Cloud sudah dibuat + billing aktif
- ☐ gcloud SDK terinstall dan login berhasil
- ☐ API App Engine, Cloud SQL, Cloud Build sudah diaktifkan
- ☐ Cloud SQL instance sudah dibuat dan aktif
- ☐ Database .sql sudah diimport ke Cloud SQL
- ☐ File app.yaml sudah dibuat dengan benar
- ☐ gcloud app deploy berhasil — URL bisa diakses

MONITORING:

- ☐ VM EC2 monitoring sudah dibuat di AWS
- ☐ Prometheus terinstall dan berjalan di port 9090
- ☐ Grafana terinstall dan berjalan di port 3000
- ☐ Prometheus berhasil scrape data dari kedua aplikasi

- ☐ Dashboard Grafana sudah menampilkan data

PENGUJIAN:

- ☐ Apache JMeter terinstall
- ☐ Skenario S-01 dijalankan di AWS — data dicatat
- ☐ Skenario S-02 dijalankan di AWS — data dicatat
- ☐ Skenario S-03 dijalankan di AWS — data dicatat
- ☐ Skenario S-04 dijalankan di AWS — data dicatat
- ☐ Skenario S-01 dijalankan di GCP — data dicatat
- ☐ Skenario S-02 dijalankan di GCP — data dicatat
- ☐ Skenario S-03 dijalankan di GCP — data dicatat
- ☐ Skenario S-04 dijalankan di GCP — data dicatat
- ☐ Screenshot Grafana diambil untuk setiap skenario

BIAYA:

- ☐ Estimasi biaya AWS sudah dihitung dari AWS Calculator
- ☐ Estimasi biaya GCP sudah dihitung dari GCP Calculator
- ☐ Screenshot/PDF hasil kalkulasi disimpan

✓ Setelah semua checklist selesai:

Buka file BAB_IV_Implementasi_Evaluasi.docx yang sudah dibuat → ganti semua angka di tabel dengan data nyata dari JMeter → update analisis sesuai temuan → Bab IV siap dikumpulkan!

TROUBLESHOOTING — MASALAH UMUM

AWS — Masalah Umum

Error: eb not recognized

→ Pastikan pip install awsebcli berhasil. Coba tutup dan buka kembali Command Prompt.

Error: 502 Bad Gateway setelah deploy

→ Jalankan eb logs untuk melihat error. Paling sering karena APP_KEY belum di-set atau koneksi database gagal.

Laravel tidak bisa konek ke RDS

→ Cek Security Group RDS apakah sudah allow inbound port 3306 dari security group Elastic Beanstalk.

GCP — Masalah Umum

Error: gcloud not recognized

→ Restart komputer setelah install Google Cloud SDK agar PATH ter-update.

Error: SQLSTATE[HY000] [2002] No such file or directory

→ Pastikan DB_SOCKET di app.yaml sudah benar (menggunakan format /cloudsql/[connection-name]).

Error 403 saat gcloud app deploy

→ Pastikan Cloud Build API sudah diaktifkan dan billing sudah terhubung ke project.

Jika masih ada masalah, copy pesan error lengkap dan cari di stackoverflow.com atau tanyakan ke dosen pembimbing.